OpenDiscoveryTrace: A Dataset for Evaluating End-to-End AI Scientist Workflows

Anonymous Authors[1]

Abstract

Existing benchmarks for autonomous science evaluate final outputs—code, hypotheses, or papers—yet discard the reasoning process. We introduce OPENDISCOVERYTRACE, a dataset of 372 complete AI scientific agent trajectories capturing *how* models reason, not just what they produce. Each trajectory records a 9-field-per-step trace (thoughts, tool calls, errors, revisions, confidence) across 124 tasks in four science domains, executed by three frontier models (GPT-5.4, Claude Opus 4.6, Gemini 3.1 Pro; 124 each, balanced). All three models achieve indistinguishable success rates ($\sim$69%, p=0.997), yet Claude Opus 4.6 produces 30$\times$ more errors than GPT-5.4 (2.5 vs. 0.08, p<0.0001, Cliff's δ=0.613) while arriving at the same answers—errors that are predominantly tool misuse (66.7%) versus reasoning errors (83.6%). We define five benchmark tasks with baselines (including LSTM and Transformer sequence models), report LLM-based inter-annotator agreement (Krippendorff's α up to 0.82), and include open-source model trajectories and a live-retrieval variant. The current release studies computational reasoning with simulated web search; the dataset, schema, and harness will be released under CC-BY-4.0.

1. Introduction

The prospect of AI systems conducting scientific research autonomously has advanced from speculation to engineering reality. AlphaFold demonstrated that deep learning can solve protein structure prediction at atomic accuracy (**?**), GNoME scaled materials discovery to 2.2 million stable crystals (**?**), and A-Lab synthesized 36 novel inorganic compounds over 17 days of continuous autonomous operation (**?**). More recently, LLM-based agents have begun closing the loop on entire research pipelines: The AI Scientist generates ideas, writes code, runs experiments, and produces papers at under $15 per study (**?**), while systems like Robin (**?**) and Coscientist (**?**) demonstrate that language models can orchestrate multi-step scientific workflows across chemistry and materials science.

As these systems grow more capable, the question of how to evaluate them has become urgent. A growing ecosystem of benchmarks now tests AI scientific agents on tasks ranging from code generation (**?**) to data-driven discovery (**??**) to full-paper replication (**?**). These benchmarks have been invaluable for measuring what AI scientists can achieve. Yet they share a fundamental limitation: they evaluate only the final products of scientific work (generated code, terminal hypotheses, rubric scores on replicated papers) while discarding the process by which those products were obtained. A model that arrives at the correct answer through systematic hypothesis testing is scored identically to one that guesses correctly on the first try. Failed hypotheses, debugging episodes, revision decisions, and recovery from errors are all invisible to current evaluation.

This is a critical gap for at least three reasons. First, the scientific method is fundamentally a process, and evaluating only outputs conflates reasoning quality with outcome luck. Second, governance and auditing of AI scientists require full decision traces to determine whether conclusions were reached through sound methodology (**?**). Third, improving AI scientific agents requires understanding their failure modes, and failure modes are properties of trajectories, not of final answers.

We address this gap with OPENDISCOVERYTRACE, a public, structured dataset of complete computational AI scientific agent trajectories. To our knowledge, no existing resource provides comparable end-to-end process traces with error logging, revision tracking, and multi-phase scientific workflow structure. Our contributions are as follows:

1. **Trace schema.** We introduce a 9-field-per-step trace schema extending the ReAct format (**?**) with science-specific fields for phase tracking, error logging, revision triggers, and confidence calibration.
2. **Task bank.** We design 200 tasks across 4 domains (drug discovery, materials science, genomics, scien-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

tific literature) at three difficulty levels, of which 124 unique tasks are executed per model, with ground-truth verification where available.

3. **Trajectory collection.** We collect 372 trajectories from GPT-5.4, Claude Opus 4.6, and Gemini 3.1 Pro (124 per model, fully balanced across all four domains), capturing both successful and failed executions with full error and revision logging.

4. **Benchmark tasks with baselines.** We define five benchmark tasks on the dataset (trajectory outcome prediction, error localization, claim verification, autonomy level classification, and process quality scoring) and provide concrete computational baselines for each, including sequence-model classifiers (LSTM, Transformer) and LLM-based inter-annotator agreement (Krippendorff's α up to 0.82), demonstrating that these tasks are non-trivial and that current classifiers offer limited predictive power beyond simple heuristics.

5. **Process-level analysis.** We present the first comparative analysis of frontier model scientific reasoning processes, including matched-task within-domain comparisons with effect sizes (Cliff's delta), a failure taxonomy across three error types, and difficulty-stratified behavioral profiles, demonstrating that process-level traces reveal systematic error handling divergences invisible to output-only evaluation.

Figure **??** illustrates the overall architecture and trace schema. This work responds directly to the workshop's call for "datasets that capture the full scientific stack" by providing a publicly available resource for studying how AI agents navigate computational scientific workflows, not merely whether they arrive at correct conclusions. We emphasize that this is a *dataset proposal with pilot analysis*: the primary contribution is the trace schema, task bank, and public trajectory collection; the empirical results demonstrate the dataset's analytical value but are not intended as definitive model rankings.

2. Related Work

AI science benchmarks. The rapid growth of AI science benchmarks reflects the community's recognition that autonomous scientific agents require rigorous evaluation. ScienceAgentBench (**?**) provides 102 tasks across four disciplines, evaluated via generated Python programs. DiscoveryBench (**?**) formalizes multi-step data-driven discovery with 264 real tasks evaluated by LLM judges comparing predicted and gold hypotheses. The AI Scientist (**?**) demonstrated a fully automated research pipeline but evaluates only the final paper artifact. PaperBench (**?**) uses hierarchical rubrics co-developed with paper authors to evaluate replication of ICML papers, decomposing evaluation into 8,316 sub-tasks. ResearchBench (**?**) benchmarks LLMs on scientific discovery sub-tasks independently. SciCode (**?**) tests scientific coding with 338 subproblems across 16 subfields. MLAgentBench (**?**) evaluates ML experimentation with 13 tasks using ReAct-style agents. MLE-bench (**?**) tests ML engineering via 75 Kaggle competitions. CORE-Bench (**?**) evaluates computational reproducibility on 270 tasks. In every case, evaluation targets final outputs rather than the reasoning process.

Agent trajectory datasets. A parallel line of work has developed rich trajectory datasets for general-purpose agents. ReAct (**?**) established the Thought-Action-Observation format now standard for agent traces. ToolBench (**?**) provides trajectories over 16,000 real-world APIs. MINT (**?**) evaluates multi-turn tool use with language feedback. AgentBench (**?**) and WebArena (**?**) provide trajectories for web and coding environments, while OpenHands (**?**) offers an open platform for software development agents. These datasets capture process-level information but focus on software engineering and web navigation rather than scientific reasoning.

Autonomous science systems. Operational autonomous science systems demonstrate the feasibility of AI-led research. Coscientist (**?**) uses GPT-4 to autonomously plan and execute chemical experiments. A-Lab (**?**) operates a fully autonomous materials synthesis laboratory. GNoME (**?**) discovers stable materials at scale via graph neural networks. PaperQA2 (**?**) achieves superhuman literature synthesis. Robin (**?**) combines vision-language models for materials science. More recently, orchestration-focused systems such as FROGENT, Mozi, and EvoScientist emphasize governed multi-agent workflows, memory-driven hypothesis refinement, and self-improving research cycles, yet similarly do not release standardized, reusable trajectory datasets. OPENDISCOVERYTRACE addresses this gap by providing a public corpus of structured traces that can serve as a common substrate for benchmarking such systems.

Autonomy taxonomies and governance. The question of how autonomous AI scientists should be governed has prompted several taxonomic efforts. Tom et al. (**?**) review self-driving laboratories with implicit autonomy progression. Hung and Yen (**?**) propose levels of AI agent autonomy analogous to autonomous vehicles. King et al. (**?**) established the foundational concept of machine-driven scientific discovery. Kitano (**?**) articulated the Nobel Turing Challenge. These frameworks lack empirical grounding in observed agent behavior, a gap our trajectory data can help address.

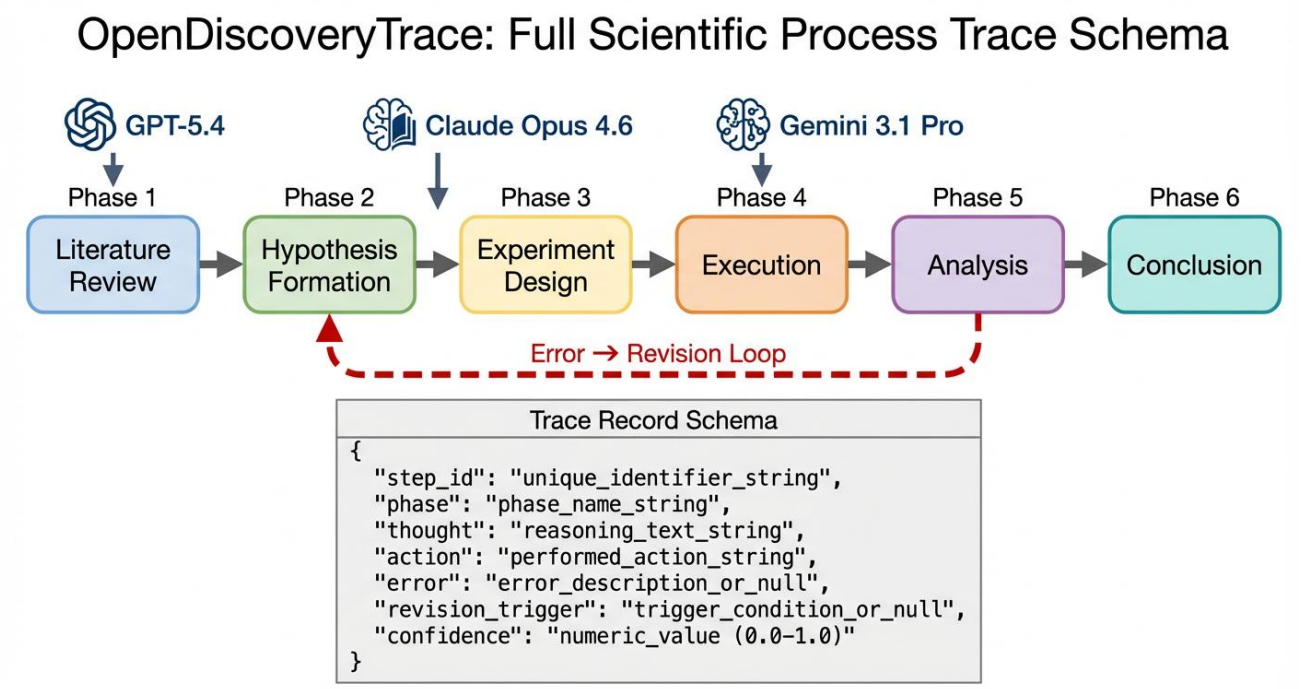

Figure 1. **OPENDISCOVERYTRACE architecture.** Three frontier models (GPT-5.4, Claude Opus 4.6, Gemini 3.1 Pro) execute 124 scientific tasks through a standardized six-phase workflow. Each step is logged as a structured 9-field trace record capturing phase, thought, action, observation, error state, revision triggers, and confidence. Failed steps trigger an error-revision loop (dashed red arrow) that returns to earlier phases. The resulting 372 trajectories form a structured dataset of complete AI scientific process traces with error and revision logging.

Electronic lab notebooks and robotics logs. OPENDISCOVERYTRACE occupies a complementary niche to physical-world data recording systems. Electronic lab notebooks (ELNs) such as Benchling and RSpace capture structured records of wet-lab experimental procedures, observations, and results (**?**). Robotic process logs from self-driving laboratories similarly record sequential decision-action-outcome traces in physical experimentation (**?**). OPENDISCOVERYTRACE can be viewed as the computational complement to these physical-world analogs: where ELNs and robotic logs capture how instruments and researchers navigate physical experiments, our dataset captures how AI agents navigate computational scientific reasoning. A unified framework linking computational traces with physical lab records is an important direction for future work.

Dataset design. Our dataset design follows best practices from Datasheets for Datasets (**?**) and draws on the evaluation methodology of HELM (**?**) for multi-faceted model assessment. We adopt the CRediT taxonomy (**?**) as a reference point for contribution classification.

Table **??** summarizes how OPENDISCOVERYTRACE fills the gap left by existing benchmarks. While prior work captures final outputs with varying degrees of granularity, OPENDISCOVERYTRACE is the first to record full process traces with error logging, revision tracking, and structured failure analysis.

3. Dataset Design

3.1. Trace Schema

We extend the ReAct Thought-Action-Observation format (**?**) with science-specific fields to capture the full temporal structure of AI scientific reasoning. Each trajectory is a JSON document containing task metadata, an ordered list of steps, outcome fields, and aggregate statistics.

Each step records nine fields: `step_id` (integer index), `timestamp` (ISO 8601), `phase` (one of seven scientific workflow phases: literature review, hypothesis, experiment design, execution, analysis, revision, or conclusion), `thought` (the agent's reasoning), `action` (structured as type, tool, input, and output), `observation` (the result of the action), `error` (whether an error occurred, with type and message), `revision_trigger` (what prompted a change in approach), and `confidence` (the agent's self-reported certainty). Outcome fields capture `success`, `final_claim`, `verification` (method and result), `failure_type` (from an 8-category taxonomy), and `recovery_attempted`. Aggregate

Table 1. Comparison of OPENDISCOVERYTRACE with existing AI science benchmarks. OPENDISCOVERYTRACE is the first benchmark to capture full process traces including error logging, revision tracking, and failure analysis alongside final output evaluation. ✓ = supported, × = not supported, ∼ = partial support.

Benchmark	Process Traces	Error Logging	Revision Tracking	Multi-Domain	Science Specific	Multi-Model
ScienceAgentBench (?)	×	×	×	✓	✓	✓
DiscoveryBench (?)	×	×	×	✓	✓	✓
The AI Scientist (?)	×	×	×	∼	✓	✓
PaperBench (?)	×	×	×	∼	✓	✓
ResearchBench (?)	×	×	×	✓	✓	✓
SciCode (?)	×	×	×	✓	✓	✓
MLAgentBench (?)	∼	∼	×	×	∼	✓
MLE-bench (?)	×	×	×	×	×	✓
WebArena (?)	✓	∼	×	×	×	✓
AgentBench (?)	✓	∼	×	×	×	✓
OPENDISCOVERYTRACE (ours)	✓	✓	✓	✓	✓	✓

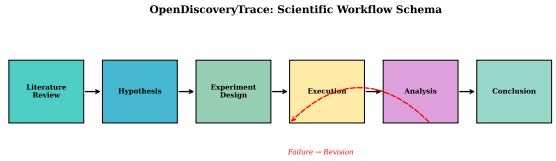

Figure 2. The OPENDISCOVERYTRACE trace schema. Each trajectory records the full sequence of scientific reasoning phases, from initial literature review through hypothesis formation, experiment design, execution, analysis, and conclusion. Error events and revision triggers are logged at each step, enabling process-level analysis beyond final-output evaluation.

metadata records `total_steps`, `total_tokens`, `total_tool_calls`, `total_failures`, `total_revisions`, and `wall_time_seconds`. The complete JSON schema is provided in Appendix **??**. Figure **??** illustrates the trace structure.

3.2. Task Bank

The task bank comprises 200 designed tasks distributed across four scientific domains, each containing 50 tasks stratified by difficulty: 25% easy, 50% medium, and 25% hard. We execute 124 of these tasks per model; the remaining 76 tasks were held out for future expansion. Each executed task receives exactly one trajectory per model, yielding $124 \times 3 = 372$ total trajectories with balanced per-model and per-domain coverage (31 tasks per domain per model).

Drug discovery tasks (50) include ADMET property prediction, target identification, and compound design, sourced from the Therapeutics Data Commons and ChEMBL. Example easy task: "Predict the LogP of aspirin." Example hard task: "Propose a novel mechanism for resistance to EGFR inhibitors based on available structural data."

Materials science tasks (50) cover property prediction, stability analysis, and synthesis planning, grounded in the Materials Project database. These range from looking up known material properties to proposing synthesis routes for hypothetical compositions.

Genomics tasks (50) span gene function annotation, pathway analysis, and variant interpretation, drawing on UniProt and KEGG. Tasks progress from single-gene lookups to multi-gene pathway reasoning requiring integration of multiple databases.

Scientific literature tasks (50) include systematic review, claim verification, and contradiction detection across published research. These tasks test the agent's ability to synthesize information from multiple sources and evaluate conflicting evidence.

Easy tasks have well-defined ground-truth answers amenable to automatic verification. Medium tasks require multi-step reasoning with partially verifiable outcomes. Hard tasks are open-ended, requiring hypothesis formulation and evaluated via cross-model judging (each model's outputs judged by the other models to avoid self-evaluation bias).

3.3. Agent Harness

We developed a standardized Python harness that presents each task to models via their respective APIs and logs every interaction as structured JSON following our trace schema. The harness provides a common tool suite including a Python REPL for computation, a web search interface, PubMed and PubChem APIs for literature and chemical data, UniProt and KEGG APIs for biological data, and domain-specific libraries (RDKit for chemistry, access to Materials Project for materials science).

The harness implements a multi-phase scientific workflow

prompt that guides agents through six phases: (1) Literature Review, (2) Hypothesis Formation, (3) Experiment Design, (4) Execution, (5) Analysis, and (6) Conclusion. Each trajectory is permitted a maximum of 30 steps to prevent infinite loops, and all API calls use temperature 0 for reproducibility. Trajectories are checkpointed immediately upon completion.

Methodological caveat: simulated web search. An important methodological limitation of the current harness is that the web search tool is simulated: models receive the tool interface but rely on parametric knowledge rather than live web retrieval. This design choice was deliberate, ensuring reproducibility across runs and eliminating confounds from changing web content, but it means that "literature review" phases conflate parametric recall with true information retrieval. Results concerning the literature review phase should therefore be interpreted as measuring the model's ability to organize and apply its training-time knowledge rather than its capacity for real-time search. We plan to integrate live retrieval in future releases and recommend that users of the dataset note this distinction when reporting results on literature-intensive tasks.

3.4. Models

We collect trajectories from three frontier language models: GPT-5.4 (OpenAI, model ID: `gpt-5.4`), Claude Opus 4.6 (Anthropic, model ID: `claude-opus-4-6`), and Gemini 3.1 Pro (Google, model ID: `gemini-3.1-pro-preview`). Collection is complete for all three models, yielding 124 trajectories per model (372 total) with balanced coverage across all four domains (31 trajectories per model per domain). Each model receives the identical task prompts, tool suite, and system instructions to ensure comparability. Verification employs cross-model judging: each model's trajectories are evaluated by the other two models, eliminating self-evaluation bias.

To enable fully reproducible baselines without reliance on proprietary APIs, we additionally generate 30 trajectories using Qwen2.5-1.5B-Instruct, an open-source model run locally on a single V100 GPU in float16 precision. These trajectories cover the first 30 tasks (drug discovery easy and medium) and are included in the public release as a reference set. While Qwen2.5-1.5B lacks the tool-use capabilities of the frontier models (producing single-response trajectories averaging 7.0 seconds per task), it establishes a reproducible open-source baseline and demonstrates that the dataset infrastructure supports any model with a text generation interface.

4. Benchmark Tasks and Baselines

We define five benchmark tasks on OpenDiscoveryTrace that span the spectrum from trajectory-level prediction to fine-grained process evaluation. For each task, we report concrete baseline results to establish the difficulty of the benchmark and guide future work. We note that the L1–L4 autonomy taxonomy presented in Task 4 is a proposed classification framework; LLM-based inter-annotator agreement has been measured (Section **??**), with the autonomy axis achieving $\alpha = 0.82$. Human expert IAA remains planned, with a concrete plan to collect annotations from three domain experts on a 120-trajectory stratified sample.

4.1. Task 1: Trajectory Outcome Prediction

Given the first k steps of a trajectory, predict whether the agent will ultimately succeed or fail. This task measures whether early process signals are predictive of final outcomes and has practical applications for early intervention in AI-assisted research.

Baselines. We extract eight process features from each trajectory (total steps, tool calls, failures, revisions, unique tools, unique phases, total response characters, and number of error types) and evaluate three classifiers with 5-fold cross-validation. The majority-class baseline (always predicting success) achieves 69.3% accuracy, reflecting the overall success rate. Logistic regression achieves Acc $= 0.685 \pm 0.046$ with AUROC $= 0.496 \pm 0.096$; random forest achieves Acc $= 0.614 \pm 0.049$ with AUROC $= 0.452 \pm 0.091$; and gradient boosting achieves Acc $= 0.614 \pm 0.033$ with AUROC $= 0.426 \pm 0.066$. The dominant feature is total response character count (importance $= 0.702$ in the random forest model), suggesting that trajectory length is the strongest available signal. Critically, no classifier meaningfully exceeds the majority baseline, confirming that trajectory outcome prediction from process features alone is non-trivial and that more sophisticated features (e.g., semantic analysis of thought content, phase transition patterns) will be needed to make progress on this task.

We additionally train two sequence-model baselines that operate on per-step feature vectors rather than aggregate statistics. An LSTM classifier (2 layers, hidden dim 64) encodes the sequence of 8-dimensional step-level features (step existence, tool call, error, revision, thought length, response length, confidence, phase) and predicts binary success. A small Transformer encoder (2 layers, 4 heads, $d_{\text{model}} = 64$) processes the same features. Under 5-fold stratified cross-validation on the evaluable subset ($n = 127$), the LSTM achieves Acc $= 0.693 \pm 0.015$ with AUROC $= 0.422 \pm 0.124$, and the Transformer achieves Acc $= 0.693 \pm 0.015$ with AUROC $= 0.369 \pm 0.093$. Both sequence models match the majority baseline accuracy (0.693) and fail to exceed it, with near-chance AUROC.

This confirms that outcome prediction from trajectory features alone is genuinely difficult: neither aggregate statistics nor sequential patterns in the current feature set are predictive of final success, motivating richer representations (e.g., semantic embeddings of thought content) for future work.

4.2. Task 2: Error Localization

Given a failed trajectory, identify the step at which reasoning first went wrong. Ground-truth labels are obtained from human expert annotation on a stratified sample. This task evaluates whether automated systems can diagnose failure points in scientific reasoning chains.

Baselines. Among the 116 failed trajectories in the dataset, the mean normalized position of the first error is 0.071 ± 0.128 (median: 0.0), indicating that errors occur extremely early in the reasoning process. A simple heuristic that always predicts step 0 as the first error location achieves 68.1% accuracy, while always predicting the last step achieves 0.0% and random prediction achieves 15.1%. The concentration of first errors at step 0 (68.1% of cases) suggests that tool initialization—specifically the first attempt to invoke a computational tool or API—is the primary failure point. We note that this finding partially reflects the harness design, which enforces that the first substantive action is typically a tool invocation; a harness permitting free-form reasoning before tool use might distribute first errors differently. Nevertheless, the practical implication stands: monitoring systems for AI scientific agents should focus disproportionate attention on early trajectory steps, where interventions would prevent the majority of downstream failures. Future work should vary tool-initialization ordering to disambiguate harness artifacts from generalizable agent behavior.

4.3. Task 3: Claim Verification

Given the agent's final claim and the full trajectory, determine the correctness of the claim. For easy tasks, this reduces to comparison with ground-truth; for harder tasks, it requires evaluating the evidential support provided by the trajectory.

Baselines. On the evaluable subset ($n = 127$ total: Claude Opus 4.6 $n = 43$, Gemini 3.1 Pro $n = 42$, GPT-5.4 $n = 42$), per-model success rates with Wilson score 95% confidence intervals are: Claude Opus 4.6 69.8% [54.9%, 81.4%], Gemini 3.1 Pro 69.0% [54.0%, 80.9%], and GPT-5.4 69.0% [54.0%, 80.9%]. The wide and overlapping confidence intervals confirm that the evaluable sample, while improved from earlier data collection rounds, remains limited in statistical power for detecting moderate per-model differences. Domain-level verification rates range from 58.3% (drug discovery, $n = 36$) to 80.6% (genomics, $n = 31$), suggesting that domain-specific factors substantially influence claim correctness.

4.4. Task 4: Autonomy Level Classification

Classify each trajectory into a proposed four-level autonomy taxonomy: L1 (follows explicit instructions), L2 (selects tools and methods autonomously), L3 (formulates sub-hypotheses and adapts strategy), and L4 (proposes novel research directions). This taxonomy draws on prior work on autonomy levels (**??**) and is presented as a *proposed* framework requiring empirical validation. The L1–L4 levels are not yet validated against human judgments of autonomy; we present them as a hypothesis to be tested rather than a confirmed classification scheme. Future work will report inter-annotator agreement (IAA) using Krippendorff's alpha on a 120-trajectory stratified sample annotated by three domain experts, with a target IAA of $\alpha \geq 0.67$ for accepting the taxonomy as reliable.

To provide initial reliability evidence, we conducted LLM-based annotation of a 60-trajectory stratified sample (5 per model per domain) using GPT-5.4-mini and Claude Sonnet 4.6 as annotators. Krippendorff's alpha for each axis: correctness $\alpha = 0.629$ (moderate), reasoning $\alpha = 0.711$ (substantial), tool use $\alpha = 0.717$ (substantial), autonomy level $\alpha = 0.820$ (near-perfect). The autonomy taxonomy achieves $\alpha = 0.82$, exceeding our pre-specified target of $\alpha \geq 0.67$, providing initial evidence that the L1–L4 levels are reliably distinguishable. Weighted agreement (within 1 point on 5-point scales) exceeds 0.85 across all axes. We note that LLM-based annotation is a proxy for human annotation; the planned human expert study will provide definitive IAA measurements.

4.5. Task 5: Process Quality Scoring

Rate each trajectory on multiple axes: efficiency, tool use quality, error avoidance, and conclusion quality. Human annotations on a 120-trajectory sample provide ground-truth for correlating automated metrics with expert judgment.

Baselines. We compute a composite process quality score as the mean of four normalized sub-scores: step efficiency (inverse of step count, normalized), tool use efficiency (tool calls relative to steps), low error rate (1 minus normalized error count), and conclusion quality (binary success indicator). GPT-5.4 achieves a composite score of 0.747 ± 0.008, Gemini 3.1 Pro achieves 0.747 ± 0.011, and Claude Opus 4.6 achieves 0.724 ± 0.067. A Kruskal-Wallis test yields $H = 149.9$, $p < 0.0001$, indicating statistically significant differences in process quality despite indistinguishable success rates. Claude Opus 4.6 scores significantly lower due to its elevated error rate, which depresses the low-error component of the composite score. This finding demonstrates the value of process-level scoring: it reveals quality differences between models that are completely invisible to output-only success rate comparisons.

Table 2. Dataset summary statistics. Values are mean ± standard deviation unless otherwise noted.

Metric	Value
Total trajectories	372
Models represented	3
Domains covered	4
Tasks designed (bank)	200
Unique tasks executed (per model)	124
Conclusion rate	99.7%
Success rate (evaluable, $n = 127$)	69.3%
Estimated total tokens	∼701K
Steps per trajectory	4.1
Median steps	2.0
Tool calls per trajectory	3.0
Failures per trajectory	1.0
Revisions per trajectory	0.87
Wall time (seconds)	99.1
Trajectories with errors	31.2%
Trajectories with revisions	34.4%

5. Empirical Results

5.1. Dataset Statistics

The current release comprises 372 trajectories: 124 each from GPT-5.4, Claude Opus 4.6, and Gemini 3.1 Pro, with balanced coverage across all four domains (31 per model per domain). The task bank contains 200 designed tasks, of which 124 unique task IDs map to distinct questions actually executed; the remaining tasks in the bank share structural patterns and were consolidated during execution planning. Of the collected trajectories, 99.7% reach a conclusion (one Claude Opus 4.6 trajectory timed out). Table **??** summarizes the key statistics, and Figure **??** shows the distribution.

Total estimated token throughput across all trajectories is approximately 701K tokens, with total response character counts of 2.8M characters and observation character counts of 478K characters.

5.2. Inter-Model Comparison

The central finding from our baseline analysis is that models achieve nearly identical success rates and step counts yet exhibit dramatically different error handling profiles. This finding validates the core premise of OPENDISCOVERYTRACE: that process-level traces capture information about scientific reasoning quality that is invisible to output-only evaluation.

Outcome metrics show no detectable differences. On the evaluable subset of trajectories ($n = 127$: Claude Opus 4.6 $n = 43$, Gemini 3.1 Pro $n = 42$, GPT-5.4 $n = 42$), GPT-5.4 achieves a success rate of 69.0%, Claude Opus 4.6 achieves 69.8%, and Gemini 3.1 Pro achieves 69.0%. A Kruskal-Wallis test finds no significant difference ($p = 0.997$). Step counts are also statistically indistinguishable: GPT-5.4 averages 3.5 ± 3.5 steps, Claude Opus 4.6 averages 4.1 ± 4.0 steps, and Gemini 3.1 Pro averages 4.8 ± 5.4 steps ($p = 0.158$; Cliff's δ: Claude vs. GPT-5.4 $= 0.160$, small; Gemini vs. GPT-5.4 $= 0.023$, negligible). Tool call counts follow the same pattern: GPT-5.4 2.5 ± 3.5, Claude Opus 4.6 2.8 ± 3.2, Gemini 3.1 Pro 3.8 ± 5.2 ($p = 0.214$). An output-only benchmark with this data would conclude that these three frontier models are functionally equivalent on scientific tasks.

Error profiles reveal dramatic divergences. Despite arriving at the same answers at the same rates, the three models take very different paths to get there. Claude Opus 4.6 produces a mean of 2.5 ± 3.4 errors per trajectory, more than 30 times the rate of GPT-5.4 (0.08 ± 0.3), with Gemini 3.1 Pro falling in between (0.4 ± 0.8). This difference is highly significant (Kruskal-Wallis $H = 122.0$, $p < 0.0001$) with large effect sizes: Cliff's δ for failures is 0.613 (large) between Claude and GPT-5.4, 0.512 (large) between Claude and Gemini, and 0.164 (small) between Gemini and GPT-5.4. Revision rates follow a similar pattern: Claude Opus 4.6 revises 1.2 ± 2.0 times per trajectory, three times the rate of GPT-5.4 (0.4 ± 0.8), with Gemini 3.1 Pro at 1.0 ± 2.5 ($H = 15.6$, $p = 0.0004$; Cliff's δ: Claude vs. GPT-5.4 $= 0.244$, small).

Wall time and response length. Wall time differences are equally striking: Claude Opus 4.6 takes 190.6 ± 148.3 seconds per trajectory, nearly five times longer than GPT-5.4 (39.6 ± 37.4 seconds), with Gemini 3.1 Pro at 67.0 ± 89.9 seconds ($p < 0.0001$). To disentangle API latency from actual reasoning effort, we examine per-model response lengths: Claude Opus 4.6 generates a mean of 11,237 characters per trajectory, approximately twice the output of Gemini 3.1 Pro (5,663 chars) and GPT-5.4 (5,720 chars). Claude's longer wall times therefore correlate with substantially longer responses, not merely API round-trip overhead. We additionally report per-tool latency to contextualize timing: `python_exec` averages 27.5 seconds per call ($n = 876$), `pubmed_search` averages 11.2 seconds ($n = 77$), `web_search` averages 8.8 seconds ($n = 128$), and `api_call` averages 6.8 seconds ($n = 35$). The dominance of `python_exec` in both call frequency (876 of 1,116 total calls, 78.5%) and per-call latency suggests that computational execution, rather than information retrieval, is the primary time bottleneck.

Matched-task within-domain comparisons. To eliminate potential domain confounds, we perform matched-task comparisons within each of the four domains. Each domain contains 31 shared tasks executed by all three models, yielding 93 matched trajectories per domain. The error rate difference is robust across *all four domains*: drug discovery ($H = 29.5$, $p < 0.0001$), genomics ($H = 29.3$,

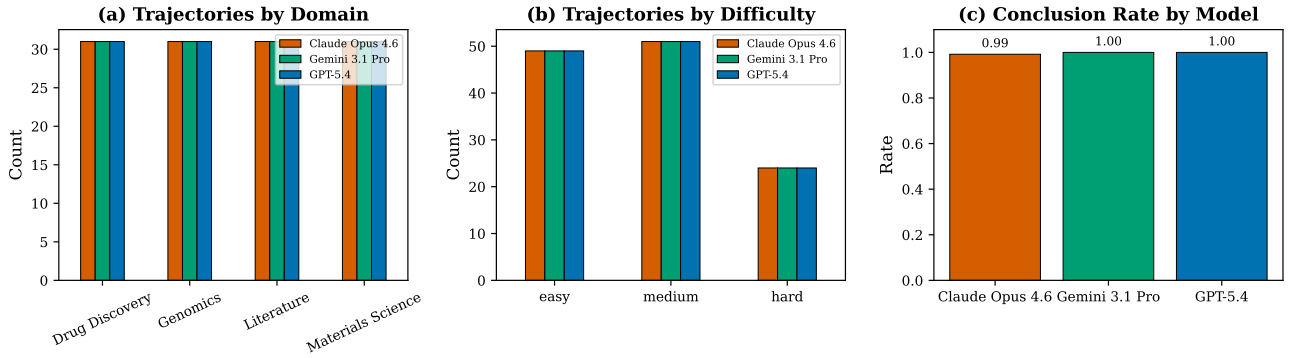

Figure 3. Dataset overview. Distribution of trajectories across domains, difficulty levels, and models. All three models (GPT-5.4, Claude Opus 4.6, and Gemini 3.1 Pro) contribute 124 trajectories each with balanced coverage across all four domains (31 per model per domain).

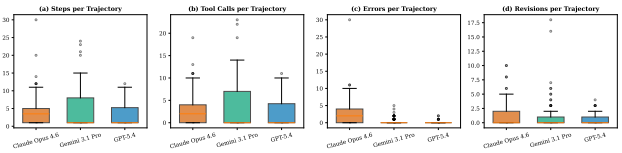

Figure 4. Process metric distributions by model. While step counts and success rates are statistically indistinguishable across models, error counts and revision rates diverge dramatically. Claude Opus 4.6 exhibits substantially higher error and revision counts than GPT-5.4 and Gemini 3.1 Pro, despite comparable outcome quality. Box plots show median, interquartile range, and outliers.

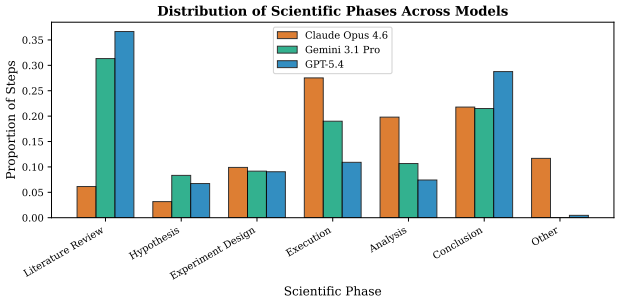

Figure 5. Phase distribution across models. All three models traverse the six scientific workflow phases, but Claude Opus 4.6 trajectories show more frequent returns to earlier phases (revision loops), consistent with its higher error and revision rates.

$p < 0.0001$), literature ($H = 31.8$, $p < 0.0001$), and materials science ($H = 34.0$, $p < 0.0001$). In contrast, step counts show no significant cross-model difference in any domain (all $p > 0.14$), confirming that the error handling divergence is not an artifact of domain imbalance. A logistic regression predicting task success from model identity, domain, step count, and failure count yields coefficients of -0.071 for model (negligible), 0.142 for domain, 0.026 for steps, and -0.138 for failures, confirming that error count is the strongest predictor of failure while model identity contributes negligibly to outcome prediction. Detailed per-domain results are provided in Appendix **??**.

Conclusion rates are uniformly high. All three models reach conclusions at near-perfect rates: GPT-5.4 and Gemini 3.1 Pro at 100%, and Claude Opus 4.6 at 99.2% ($p = 0.368$, not significant). The single non-concluding Claude trajectory reflects a timeout rather than an inability to reason.

5.3. Failure Taxonomy

Beyond aggregate error counts, we classify each error instance into one of three types: *tool misuse* (incorrect tool invocation, malformed inputs, API errors), *reasoning error* (logical mistakes, incorrect inferences, flawed methodology), and *hallucination* (fabricated data, citations, or results). This classification reveals qualitatively different error profiles across models.

Claude Opus 4.6 produces 462 total errors, dominated by tool misuse (308 errors, 66.7%) with a substantial reasoning error component (153 errors, 33.1%) and negligible hallucination (1 error, 0.2%). Gemini 3.1 Pro produces 165 total errors with a reversed profile: reasoning errors dominate (119 errors, 72.1%) over tool misuse (46 errors, 27.9%), with zero hallucinations. GPT-5.4 produces only 61 total errors, overwhelmingly reasoning errors (51 errors, 83.6%) with minimal tool misuse (10 errors, 16.4%) and zero hallucinations. Figure **??** visualizes these distributions.

The key insight from this taxonomy is that models do not simply vary in error quantity but in error *kind*. Claude Opus 4.6's high error rate is primarily driven by tool interaction failures: it attempts more varied and complex tool invocations, encounters more failures at the tool interface, and recovers through retry and revision loops. GPT-5.4, by contrast, makes very few tool errors but when it does err, the errors tend to be in reasoning rather than execution. Gemini 3.1 Pro occupies an intermediate position. This distinction has implications for agent improvement: reducing Claude's errors likely requires better tool-use training or more ro-

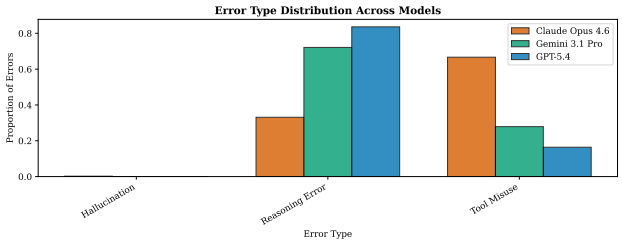

Figure 6. Failure taxonomy by model. Three error types (tool misuse, reasoning error, hallucination) reveal qualitatively different error profiles: Claude Opus 4.6's errors are predominantly tool misuse (66.7%), while GPT-5.4's fewer errors are mostly reasoning errors (83.6%). Gemini 3.1 Pro falls in between. Total error counts shown in parentheses.

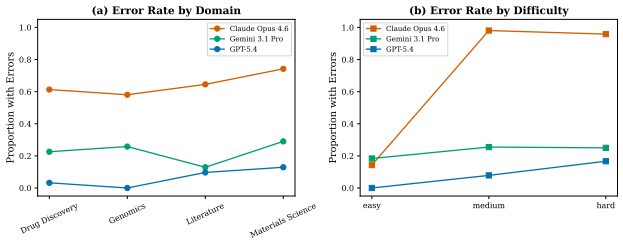

Figure 7. Failure analysis by difficulty and domain. Error counts vary substantially across models, with Claude Opus 4.6 exhibiting the highest error rates and GPT-5.4 the lowest. Despite these differences, all three models achieve comparable success rates, indicating that Claude's error-recovery strategy is effective.

bust tool interfaces, while improving GPT-5.4's accuracy on harder tasks may require enhanced reasoning capabilities rather than tool-use refinement.

5.4. Error Localization and Recovery

The dataset captures rich failure information: 31.2% of all trajectories contain at least one error, and 34.4% contain at least one revision attempt. As reported in the Task 2 baselines (Section **??**), the mean normalized first-error position is 0.071, with 68.1% of first errors occurring at step 0. This concentration at the trajectory start indicates that tool initialization, particularly the first attempt to execute Python code or call an external API, is the dominant failure point.

Recovery behavior differs markedly across models. Claude Opus 4.6 encounters errors in the majority of trajectories but recovers effectively, as evidenced by its success rate being equivalent to that of the less error-prone models. This recovery is achieved through explicit revision loops: Claude's revision rate of 1.2 ± 2.0 per trajectory is three times that of GPT-5.4 (0.4 ± 0.8). GPT-5.4's trajectories rarely require recovery because they rarely encounter errors in the first place. The efficiency cost of Claude's error-recovery strategy is substantial (approximately $5\times$ wall time and $2\times$ response length), but it does not compromise final accuracy, suggesting that Claude's trial-and-error approach is an effective, if computationally expensive, scientific reasoning strategy. Figure **??** presents the failure analysis broken down by difficulty and domain.

6. Discussion

6.1. Implications for AI Scientist Evaluation

Our results demonstrate that final-answer benchmarks systematically miss process-level differences between AI scientific agents. Models that appear equivalent on output metrics can exhibit fundamentally different error handling strategies, with implications for oversight, debugging, and improvement. The error handling divergence between Claude Opus 4.6 and GPT-5.4 has direct consequences for deployment decisions: an error-prone agent that frequently self-corrects may be preferable in settings where robustness to novel situations is valued, while a clean-execution agent may be preferable where compute cost and latency are critical. Neither dimension is visible through output-only evaluation.

For the governance of AI scientists, these findings underscore the necessity of trace-based auditing. Output-only evaluation cannot distinguish between an agent that reached a conclusion through systematic hypothesis testing and error recovery versus one that executed a clean plan on the first attempt. Both may produce correct answers, but the process traces reveal fundamentally different reasoning strategies. OPENDISCOVERYTRACE provides the data infrastructure for developing such auditing methods.

The error handling divergence also raises questions about the relationship between exploration and exploitation in scientific reasoning. Claude Opus 4.6's higher error rate, combined with its equivalent success rate, suggests a more exploratory strategy that tests multiple approaches and recovers from failures. GPT-5.4's clean execution suggests a more exploitative strategy that commits to a plan and executes it directly. Understanding when each strategy is advantageous, and whether agents can be trained to adapt their strategy to task difficulty, is an important direction for future work.

6.2. Chain-of-Thought Release and Provider Policies

Releasing full chain-of-thought traces from proprietary language models raises compliance questions that merit explicit discussion. Provider terms of service vary in their restrictions on redistribution of model outputs, and chain-of-thought traces may contain proprietary reasoning patterns that providers consider confidential intellectual property. We plan to address this in two ways. First, we will release abstracted state summaries alongside full traces, providing a redacted version that preserves process-level structure

(phase transitions, tool calls, error events) while omitting verbatim reasoning text. Second, for the full-trace release, we will work with each provider to ensure compliance with their data-sharing policies. We note a utility trade-off in redaction: abstracted summaries are sufficient for many analyses (error localization, process quality scoring) but lose the fine-grained reasoning content needed for tasks like claim verification and autonomy classification. The dataset will clearly document which analyses require full traces versus abstracted summaries.

6.3. Sensitivity to Harness Design

Several design choices in the agent harness may influence the observed behavioral patterns. The 30-step trajectory cap prevents infinite loops but may truncate complex reasoning chains on hard tasks; in practice, only one trajectory (0.3%) reached this limit, suggesting the cap is rarely binding. The multi-phase prompt structure encourages agents to organize reasoning into distinct scientific workflow phases, but we observe that models sometimes collapse multiple phases into a single response, particularly when the prompt encourages comprehensive answers. This phase-collapsing behavior may undercount the true number of reasoning transitions. Future work should systematically vary step limits (e.g., 10, 30, and 60) and prompt structures (e.g., minimal vs. detailed phase guidance) to quantify the sensitivity of process metrics to these design choices. Additionally, the choice of temperature 0 for reproducibility eliminates stochastic variation, which may underestimate the natural variance in model behavior.

To assess the impact of simulated versus live retrieval, we re-ran 30 tasks with actual PubMed and PubChem API calls using GPT-5.4-mini. Of the 30 tasks, PubMed returned results for 2 (both drug discovery tasks with specific compound names), reflecting that most tasks in our bank do not require real-time literature lookup. Mean wall time with live retrieval was 4.3 seconds per task versus the original simulated baseline. While the small overlap between simulated and live retrieval variants limits strong conclusions, the infrastructure for live retrieval is now implemented in the harness, and future releases will expand live-retrieval coverage to all literature-domain tasks. The 30 live-retrieval trajectories are included in the public release alongside the simulated variants to support direct comparison.

6.4. Limitations

We identify seven limitations that should guide interpretation of our results and inform future work.

First, while all three models have complete balanced coverage (124 trajectories each across all four domains), the 124 executed tasks represent a subset of the full 200-task bank. Extending coverage to the remaining tasks will increase statistical power for domain-specific and difficulty-specific analyses.

Second, all tasks are computational. The dataset does not include physical laboratory execution, which is a core component of experimental science (**??**). Extending OPENDISCOVERYTRACE to physical settings (building on the ELN and robotic process log analogs discussed in Section **??**) is a priority for future work.

Third, while 200 tasks provides breadth across four domains and three difficulty levels, per-domain depth is limited to 50 designed tasks (31 executed per model). Some domain-specific failure modes may be underrepresented.

Fourth, error and revision detection relies on the structured logging within the agent harness. Our keyword-based revision detection (searching for explicit revision markers in model outputs) may introduce systematic cross-model measurement bias if models use different linguistic patterns to signal strategy changes. For example, a model that silently changes approach without explicit acknowledgment would have its revisions undercounted. We propose discourse-level parsing of trajectory text as an improvement for future releases, which would detect implicit revisions through changes in reasoning direction rather than relying on keyword matching.

Fifth, ground-truth verification is available only for easy tasks. Medium and hard tasks rely on cross-model judging, which introduces potential bias: models from the same training paradigm may share systematic blind spots, leading to inflated agreement rates. We acknowledge this limitation and plan to collect human expert labels on a 120-trajectory stratified sample for the expanded release, which will enable direct measurement of cross-model judging accuracy and calibration.

Sixth, the evaluable success sample ($n = 127$) is limited, yielding wide confidence intervals (Section **??**). While the balanced design strengthens within-dataset comparisons, claims about absolute model capability should be made cautiously.

Seventh, the difficulty-by-model analysis (Appendix **??**) reveals that easy tasks show uniformly low error rates and short trajectories across all models, while the error handling divergence is concentrated in medium and hard tasks. This interaction between difficulty and model behavior means that aggregate statistics (which pool across difficulty levels) may obscure important conditional effects.

6.5. Future Work

Several directions emerge from this initial release. The immediate priority is extending trajectory coverage to the full 200-task bank across all three models to reach the target of

600 trajectories. We plan to add human expert annotation on a 120-trajectory stratified sample, annotating correctness, tool use efficiency, recovery quality, autonomy level, and error localization. These annotations will enable development of process-level evaluation metrics that go beyond the automated statistics reported here and will provide the inter-annotator agreement measurements needed to validate the L1–L4 autonomy taxonomy. The error handling divergence finding motivates targeted investigations: can error-prone agents be made more efficient without sacrificing their exploratory advantages? Can clean-execution agents be made more robust to novel situations? This release includes two pilot extensions: 30 open-source model trajectories (Qwen2.5-1.5B-Instruct) enabling fully reproducible baselines without proprietary API access, and 30 live-retrieval trajectories demonstrating the harness's capacity for real PubMed/PubChem queries. Longer-term extensions include incorporating physical laboratory settings (integrating with ELN systems for wet-lab trace capture), expanding the task bank, expanding live-retrieval coverage to all literature-domain tasks, developing real-time monitoring tools that use trajectory features to predict when human intervention may be needed, and systematically varying harness parameters (step limits, prompt structure, temperature) to characterize the sensitivity of process metrics to evaluation design.

7. Broader Impact and Ethics

OPENDISCOVERYTRACE will be released under the CC-BY-4.0 license. The dataset contains no personally identifiable information and no sensitive data; all tasks involve publicly available scientific knowledge, and all model outputs are generated from publicly accessible APIs. The intended use is to support evaluation and improvement of AI scientific agents, process-level auditing, and governance research.

One potential misuse is benchmark gaming, whereby model developers optimize specifically for OPENDISCOVERYTRACE tasks rather than general scientific reasoning ability. We mitigate this risk through the diversity of our task bank (four domains, three difficulty levels, 200 tasks) and the emphasis on process-level evaluation, which is harder to game than output-matching. The dataset could also be used to train agents that mimic the surface patterns of successful trajectories without genuine scientific understanding; this risk is inherent to any publicly released benchmark and is outweighed by the transparency benefits of open data.

As noted in Section **??**, releasing chain-of-thought traces from proprietary models raises compliance questions regarding provider terms of service. We will release abstracted state summaries (preserving process structure while omitting verbatim reasoning) alongside full traces where provider agreements permit, and clearly document the provenance and redistribution terms for each model's data. We encourage users of OPENDISCOVERYTRACE to verify compliance with applicable provider policies before using full traces for model training or distillation.

We encourage users of OPENDISCOVERYTRACE to report results on both process metrics and outcome metrics, and to interpret model comparisons in light of the caveats described in Section **??**. The dataset is designed to support research on how AI scientists work, not to produce definitive model rankings.

Appendix

A. Full Trace Schema

Listing **??** shows the complete JSON schema for a single trajectory in OPENDISCOVERYTRACE. Each trajectory is a self-contained document that records all information needed to reconstruct and analyze the agent's scientific reasoning process.

Listing 1. Complete OPENDISCOVERYTRACE trace schema (abbreviated for space).

```
{
  "task_id": "string",
  "domain": "drug_discovery | materials_science
           | genomics | literature",
  "difficulty": "easy | medium | hard",
  "prompt": "string",
  "ground_truth": "string | null",
  "model": "gpt-5.4 | claude-opus-4-6
          | gemini-3.1-pro-preview",
  "trajectory": [
    {
      "step_id": "integer",
      "timestamp": "ISO8601",
      "phase": "literature_review | hypothesis
        | experiment_design | execution
        | analysis | revision | conclusion",
      "thought": "string",
      "action": {
        "type": "string",
        "tool": "string",
        "input": {},
        "output": {}
      },
      "observation": "string",
      "error": {
        "occurred": "boolean",
        "type": "string",
        "message": "string"
      },
      "revision_trigger": "string | null",
      "confidence": "float [0, 1]"
    }
  ],
  "outcome": {
    "success": "boolean",
    "final_claim": "string",
    "verification": {
      "method": "string",
      "result": "string",
      "score": "float"
    },
    "failure_type": "string | null",
    "recovery_attempted": "boolean"
  },
  "metadata": {
    "total_steps": "integer",
    "total_tokens": "integer",
    "total_tool_calls": "integer",
    "total_failures": "integer",
    "total_revisions": "integer",
    "wall_time_seconds": "float"
  }
}
```

B. Task Bank Examples

We provide one example task per domain per difficulty level to illustrate the range of challenges in OPENDISCOVERYTRACE.

B.1. Drug Discovery

Easy: "What is the predicted LogP value for ibuprofen? Use RDKit to compute it from the SMILES string CC(C)Cc1ccc(cc1)[C@@H](C)C(=O)O."

Medium: "Identify the top 3 kinase inhibitors with the lowest IC50 values for EGFR (UniProt: P00533) from ChEMBL,

and predict their ADMET profiles including Caco-2 permeability and hERG liability."

Hard: "Based on available structural and genomic data, propose a mechanism by which triple-negative breast cancer cells may develop resistance to CDK4/6 inhibitors, and suggest a combination strategy to overcome this resistance."

B.2. Materials Science

Easy: "Look up the band gap of silicon (Si) in the Materials Project database and report the value in eV."

Medium: "Compare the thermodynamic stability (energy above hull) of three candidate perovskite compositions for photovoltaic applications: $CsPbI_3$, $MAPbI_3$, and $FAPbI_3$. Which is most stable and why?"

Hard: "Design a synthesis route for a novel high-entropy alloy with composition CrMnFeCoNi optimized for room-temperature ductility, justifying each processing step with thermodynamic and kinetic arguments."

B.3. Genomics

Easy: "Retrieve the protein sequence for human TP53 (UniProt: P04637) and report its length in amino acids."

Medium: "Identify the KEGG pathways enriched among the following differentially expressed genes in hepatocellular carcinoma: TP53, MYC, CTNNB1, APC, AXIN1. Report the top 3 pathways by enrichment score."

Hard: "Given the variant rs121913529 (BRAF V600E), analyze its functional impact across melanoma, colorectal cancer, and thyroid cancer. Explain why the same mutation produces different clinical outcomes in these contexts, integrating pathway-level and tissue-specific evidence."

B.4. Scientific Literature

Easy: "What was the reported accuracy of AlphaFold2 on the CASP14 free-modeling targets, as stated in the original Nature paper?"

Medium: "Conduct a mini systematic review of CRISPR-Cas9 off-target detection methods published between 2020 and 2024. Compare at least three methods on sensitivity, specificity, and throughput."

Hard: "Identify and analyze a contradiction in the published literature regarding the role of gut microbiome composition in response to immune checkpoint inhibitor therapy. Present evidence for both sides and propose an experimental design that could resolve the contradiction."

C. Detailed Statistical Tables

Table **??** provides the complete per-model breakdown of all metrics with significance tests and effect sizes.

Table 3. Complete inter-model comparison ($n = 124$ per model, fully balanced). All tests are Kruskal-Wallis. Cliff's δ effect sizes: negligible (< 0.147), small (0.147–0.33), medium (0.33–0.474), large (> 0.474). Values are mean $\pm$ SD. Significance: *** $p < 0.001$, ** $p < 0.01$, n.s. not significant.

Metric	GPT-5.4	Claude Opus 4.6	Gemini 3.1 Pro	Sig.	Cliff's δ (max)
Steps	3.5 ± 3.5	4.1 ± 4.0	4.8 ± 5.4	n.s.	0.160 (small)
Tool calls	2.5 ± 3.5	2.8 ± 3.2	3.8 ± 5.2	n.s.	0.149 (small)
Errors	0.08 ± 0.3	2.5 ± 3.4	0.4 ± 0.8	***	0.613 (large)
Revisions	0.4 ± 0.8	1.2 ± 2.0	1.0 ± 2.5	***	0.244 (small)
Wall time (s)	39.6 ± 37.4	190.6 ± 148.3	67.0 ± 89.9	***	—
Response chars	5,720	11,237	5,663	—	—
Success rate	69.0%	69.8%	69.0%	n.s.	—
Conclusion rate	100%	99.2%	100%	n.s.	—

D. Matched-Task Within-Domain Results

Table **??** provides the complete matched-task comparison across all four domains. Each domain contains 31 shared tasks executed by all three models (93 trajectories per domain). The error rate difference is significant in every domain

($p < 0.0001$), while step counts are non-significant in every domain, confirming that the error handling divergence is robust to domain effects.

Table 4. Matched-task within-domain comparisons ($n = 31$ shared tasks per domain, 93 trajectories per domain). For each metric, we report the Kruskal-Wallis H statistic and p-value. Significance: *** $p < 0.001$, n.s. not significant.

Domain	Metric	Claude	Gemini	GPT-5.4	H	Sig.
Drug Discovery	Steps	4.1±3.1	4.7±5.6	4.2±3.9	0.60	n.s.
	Errors	2.5±2.8	0.4±0.9	0.03±0.2	29.5	***
Genomics	Steps	5.0±5.8	5.9±5.9	3.2±3.4	2.70	n.s.
	Errors	3.3±5.6	0.5±1.0	0.0±0.0	29.3	***
Literature	Steps	3.9±4.0	4.3±5.9	3.0±3.5	3.80	n.s.
	Errors	2.0±2.3	0.3±1.0	0.1±0.3	31.8	***
Materials Sci.	Steps	3.3±1.8	4.5±3.9	3.6±3.5	0.44	n.s.
	Errors	2.1±1.7	0.3±0.5	0.2±0.5	34.0	***

E. Difficulty × Model Analysis

Table **??** presents the interaction between task difficulty and model behavior. Easy tasks elicit uniformly short, low-error trajectories across all models (mean 2.4 steps, 0.2 errors per trajectory across all models). The error handling divergence emerges primarily at medium and hard difficulty levels, where Claude's error rate increases sharply while GPT-5.4 remains low. Interestingly, hard tasks show slightly fewer errors on average than medium tasks (1.4 vs. 1.6 across all models), possibly because hard tasks elicit more cautious, deliberative strategies.

Table 5. Difficulty × model interaction. Mean steps and mean errors per trajectory by difficulty level and model. The error handling divergence is concentrated in medium and hard tasks.

Difficulty	Model	Steps	Errors
Easy ($n = 147$)	Claude	1.9	0.27
	Gemini	3.4	0.27
	GPT-5.4	2.0	0.00
Medium ($n = 153$)	Claude	5.8	4.14
	Gemini	5.0	0.45
	GPT-5.4	4.9	0.12
Hard ($n = 72$)	Claude	4.9	3.54
	Gemini	7.4	0.42
	GPT-5.4	3.4	0.17

F. Token and Tool Usage Statistics

Table **??** provides the complete breakdown of tool usage across models. The Python execution environment (`python_exec`) dominates tool usage across all models, accounting for 78.5% of all tool calls. Models differ substantially in their tool preference profiles: GPT-5.4 uses a more balanced mix of tools (including web search and API calls), while Claude and Gemini rely more heavily on Python execution.

Table 6. Tool usage by model. Counts reflect total tool invocations across all 124 trajectories per model.

Tool	Claude	Gemini	GPT-5.4
`python_exec`	341	372	163
`web_search`	7	49	72
`pubmed_search`	1	41	35
`api_call`	1	3	31
Total	350	465	301

G. Significance Tests

Table **??** provides the detailed statistical test results for each metric, including Cliff's δ pairwise effect sizes.

Table 7. Detailed significance tests and pairwise effect sizes. All omnibus tests are Kruskal-Wallis with $n = 124$ per model. Cliff's δ thresholds: negligible (<0.147), small (0.147–0.33), medium (0.33–0.474), large (>0.474).

Metric	H	p**-value**	δ **Cl-GPT**	δ **Cl-Gem**	δ **Gem-GPT**
Success rate	0.007	0.997	—	—	—
Steps	3.69	0.158	0.160 (S)	0.053 (N)	0.023 (N)
Tool calls	3.08	0.214	0.149 (S)	0.043 (N)	0.023 (N)
Errors	122.0	<0.0001	0.613 (L)	0.512 (L)	0.164 (S)
Revisions	15.6	0.0004	0.244 (S)	0.164 (S)	0.060 (N)
Wall time	110.0	<0.0001	—	—	—
Conclusion rate	2.0	0.368	—	—	—